Nexora Technologies — Backend Architecture & API Contract

Software Design Document (SDD) / Low-Level Design (LLD)

Document Type	Internal Backend Specification
System	Nexora Technologies Corporate Website — Backend
Stack	Java 21, Spring Boot 3.x, Spring Data JPA, MySQL 8, Spring Validation, Spring Async, Lombok
Audience	Backend Engineers, Frontend Engineers, QA Engineers, Future Team Members
Status	Draft v1.0 — official spec, implement exactly as written unless a change is agreed and this document is updated
Frontend Reviewed	`ApplicationForm.jsx`, `ProgramApplicationForm.jsx`, `ProgramInfoStep.jsx`, `ConsentStep.jsx`, `ProgramStepper.jsx`, `SuccessScreen.jsx`, `ProgramSuccessScreen.jsx`, `CareersLayout.jsx`, `CareersPage.jsx`, `JobPage.jsx`, `ProgramPage.jsx`, `LandingPage.jsx`, `jobsData.js`

> *This document is the source of truth for the backend. It was produced by analyzing the actual React components — not just the feature list — so field names, steps, and button labels below map 1:1 to what the frontend already renders. Section 31 calls out three places where the current frontend code disagrees with this contract; those are the only frontend changes this document asks for.*

Table of Contents

31. Frontend Code Observations

1. High-Level Architecture

DIAGRAM (MERMAID SOURCE)

```
flowchart LR
    subgraph Client["React Frontend (SPA)"]
        A[Landing Page]
        B[Careers Page]
        C[Job Page]
        D[ASE Program Page]
        E[Footer / Contact]
    end

    subgraph Edge["Edge / Infra"]
        LB[Load Balancer / Reverse Proxy]
    end

    subgraph Backend["Spring Boot Backend"]
        CTRL[Controller Layer]
        SVC[Service Layer]
        REPO[Repository Layer - Spring Data JPA]
        ASYNC[Async Email Executor]
    end

    subgraph Data["Data Stores"]
        DB[(MySQL)]
        S3[(Object Storage - Resumes)]
    end

    subgraph External["External Services"]
        SMTP[Email Provider - SMTP / SES / SendGrid]
    end

    Client -->|HTTPS / REST + JSON / multipart| LB --> CTRL
    CTRL --> SVC
    SVC --> REPO --> DB
    SVC -->|upload/read resume| S3
    SVC --> ASYNC --> SMTP
```

Style: Layered monolith (Controller → Service → Repository → Entity), single deployable Spring Boot application, single MySQL schema. This is intentional — the feature set (careers, contact, newsletter) does not justify microservices at this stage. Module boundaries inside the monolith (Section 2) are drawn so that splitting into services later, if ever needed, is a mechanical extraction rather than a rewrite.

2. Module Breakdown

The frontend has many *pages*. The backend organizes around **business domains**, not pages — a page is a UI concern, a domain is a data-ownership concern. There are 4 backend modules:

Module	Owns	Frontend Pages That Call It
Careers	Jobs, Program Applications, Job Applications	`CareersPage`, `JobPage`, `ProgramPage`
Contact	Callback Leads, Consultation Leads	`LandingPage` (contact section)
Newsletter	Subscribers	`LandingPage` / `CareersLayout` footer
Admin *(future)*	Everything above, in read/write admin form	Future Admin Portal (separate app)

Each module is a vertical slice: its own controller(s), service(s), repository, entity/entities, DTOs, and mapper. Modules do not reach into each other's repositories — if Admin needs Career data, it goes through `CareerService`, not `ApplicationRepository` directly.

3. Folder Structure

```
nexora-backend/
├── src/main/java/com/nexora/backend/
│   ├── NexoraBackendApplication.java
│   │
│   ├── config/
│   │   ├── AsyncConfig.java            # @EnableAsync, thread pool for email
│   │   ├── CorsConfig.java             # allowed origins for the React app
│   │   ├── OpenApiConfig.java          # Swagger/OpenAPI bean
```

```
│   │   ├── FileStorageConfig.java      # S3 client bean, bucket/region props
│   │   └── WebConfig.java              # multipart limits, jackson config
│   │
│   ├── common/
│   │   ├── dto/
│   │   │   ├── ApiResponse.java        # generic success/error envelope
│   │   │   └── PageResponse.java       # generic paginated envelope
│   │   ├── exception/
│   │   │   ├── GlobalExceptionHandler.java
│   │   │   ├── ResourceNotFoundException.java
│   │   │   ├── InvalidStateException.java   # e.g. submit on already-SUBMITTED app
│   │   │   ├── FileValidationException.java
│   │   │   └── ErrorCode.java
│   │   └── util/
│   │       ├── ApplicationIdGenerator.java
│   │       └── FileValidationUtil.java
│   │
│   ├── career/
│   │   ├── controller/
│   │   │   ├── JobController.java
│   │   │   ├── ProgramApplicationController.java
│   │   │   └── JobApplicationController.java
│   │   ├── service/
│   │   │   ├── JobService.java
│   │   │   ├── ProgramApplicationService.java
│   │   │   └── JobApplicationService.java
│   │   ├── repository/
│   │   │   ├── ApplicationRepository.java
│   │   │   └── JobRepository.java
│   │   ├── entity/
│   │   │   ├── Application.java
│   │   │   └── Job.java
│   │   ├── enums/
│   │   │   ├── ApplicationType.java
│   │   │   ├── ApplicationStatus.java
│   │   │   ├── JobStatus.java
│   │   │   └── EmploymentType.java
│   │   ├── dto/
│   │   │   ├── request/
│   │   │   │   ├── ProgramApplicationCreateRequest.java
│   │   │   │   ├── ProgramApplicationSubmitRequest.java
│   │   │   │   └── JobApplicationCreateRequest.java
│   │   │   └── response/
│   │   │       ├── ApplicationCreatedResponse.java
│   │   │       ├── ApplicationSubmittedResponse.java
│   │   │       └── JobResponse.java
│   │   └── mapper/
│   │       ├── ApplicationMapper.java
│   │       └── JobMapper.java
│   │
│   ├── contact/
│   │   ├── controller/ContactController.java
│   │   ├── service/ContactService.java
│   │   ├── repository/LeadRepository.java
│   │   ├── entity/Lead.java
│   │   ├── enums/{LeadType.java, LeadStatus.java}
│   │   ├── dto/request/{CallbackRequest.java, ConsultationRequest.java}
│   │   └── mapper/LeadMapper.java
│   │
│   ├── newsletter/
│   │   ├── controller/NewsletterController.java
│   │   ├── service/NewsletterService.java
│   │   ├── repository/NewsletterSubscriberRepository.java
│   │   ├── entity/NewsletterSubscriber.java
│   │   ├── enums/NewsletterStatus.java
│   │   └── dto/request/NewsletterSubscribeRequest.java
│   │
│   ├── email/
│   │   ├── EmailService.java           # sendApplicationReceivedEmail(), etc.
│   │   ├── EmailTemplateService.java   # renders Thymeleaf/Freemarker templates
│   │   └── templates/*.html
│   │
│   ├── storage/
│   │   ├── ResumeStorageService.java   # interface
│   │   └── S3ResumeStorageServiceImpl.java
│   │
│   └── admin/                          # FUTURE — package placeholder only
│       └── (see Section 20)
```

```
│
├── src/main/resources/
│   ├── application.yml
│   ├── application-dev.yml
│   ├── application-prod.yml
│   └── db/migration/                  # Flyway scripts, V1__init.sql ...
│
└── src/test/java/com/nexora/backend/   # mirrors main package structure
```

Why this shape: `common` holds only truly cross-cutting code (error envelope, exceptions). Every business module is self-contained under its own package with controller → service → repository → entity → dto → mapper, so a new engineer can open one folder and understand one feature completely, and the `admin` module can be added later without touching existing packages.

---

4. Backend Architecture Diagram

DIAGRAM (MERMAID SOURCE)

```
flowchart TB
    subgraph L1["Controller Layer"]
        direction LR
        C1[JobController]
        C2[ProgramApplicationController]
        C3[JobApplicationController]
        C4[ContactController]
        C5[NewsletterController]
    end

    subgraph L2["Service Layer"]
        direction LR
        S1[JobService]
        S2[ProgramApplicationService]
        S3[JobApplicationService]
        S4[ContactService]
        S5[NewsletterService]
        S6[EmailService - async]
        S7[ResumeStorageService]
    end

    subgraph L3["Repository Layer"]
        direction LR
        R1[JobRepository]
        R2[ApplicationRepository]
        R3[LeadRepository]
        R4[NewsletterSubscriberRepository]
    end

    subgraph L4["Persistence"]
        DB[(MySQL)]
        S3B[(S3 Bucket)]
    end

    C1 --> S1 --> R1 --> DB
    C2 --> S2 --> R2 --> DB
    C3 --> S3 --> R2
    C4 --> S4 --> R3 --> DB
    C5 --> S5 --> R4 --> DB
    S2 --> S6
    S3 --> S6
    S4 --> S6
    S2 --> S7 --> S3B
    S3 --> S7
```

Each layer has one direction of dependency: Controller → Service → Repository. Controllers never call repositories directly, and services never build HTTP-specific objects (`ResponseEntity`, request params) — that stays in the controller.

---

5. Frontend ↔ Backend Flow Diagram

DIAGRAM (MERMAID SOURCE)

```
flowchart TD
    subgraph ASE["ASE Program (3 backend-relevant steps)"]
        A1["Step 1: Basic Details
ProgramApplicationForm.jsx"] -->|POST program/applications| A2["Application row created
status=PENDING"]
        A2 --> A3["Step 2: Program Info
ProgramInfoStep.jsx
(no API call)"]
        A3 --> A4["Step 3: Consent
ConsentStep.jsx"]
        A4 -->|PATCH .../submit| A5["status=SUBMITTED
email sent async"]
        A5 --> A6["ProgramSuccessScreen.jsx"]
    end

    subgraph JOB["Normal Job Application (1 step)"]
        B1["ApplicationForm.jsx"] -->|POST jobs/id/applications| B2["Application row created
status=SUBMITTED directly
email sent async"]
        B2 --> B3["SuccessScreen.jsx"]
    end

    subgraph CONTACT["Contact"]
        D1["Request Callback"] -->|POST contact/callback| D2["Lead created
leadType=CALLBACK"]
        D3["Schedule Consultation"] -->|POST contact/consultation| D4["Lead created
leadType=CONSULTATION"]
    end

    subgraph NEWS["Newsletter"]
        E1["Footer form"] -->|POST newsletter/subscribe| E2["Subscriber created/reactivated"]
    end
```

---

6. Database ER Diagram

DIAGRAM (MERMAID SOURCE)

```
erDiagram
    JOB ||--o{ APPLICATION : "receives (nullable for PROGRAM type)"

    JOB {
        bigint id PK
        varchar slug UK
        varchar title
        varchar department
        varchar experience
        varchar location
        varchar employment_type
        text description
        varchar status
        datetime created_at
        datetime updated_at
    }

    APPLICATION {
        varchar id PK "e.g. APP202600001"
        varchar application_type
        bigint job_id FK "NULL for PROGRAM"
        varchar job_title_snapshot
        varchar full_name
        varchar email
        varchar phone
        varchar college "NULL for JOB"
        int graduation_year "NULL for JOB"
        varchar resume_url
        varchar resume_original_name
        varchar resume_mime_type
        bigint resume_size_bytes
        varchar resume_storage_key
        text note "JOB only, optional"
        varchar status
        boolean is_consent
        datetime submitted_at "NULL until SUBMITTED"
        datetime created_at
        datetime updated_at
    }

    LEAD {
        bigint id PK
        varchar lead_type
        varchar full_name
        varchar company "NULL for CALLBACK"
        varchar email
        varchar phone
        varchar project_type "NULL for CALLBACK"
        varchar budget "NULL for CALLBACK"
        text message "NULL for CALLBACK"
        varchar status
        datetime created_at
        datetime updated_at
    }

    NEWSLETTER_SUBSCRIBER {
        bigint id PK
        varchar email UK
        varchar status
        datetime subscribed_at
        datetime unsubscribed_at "NULL while ACTIVE"
    }
```

No FK exists between `LEAD` / `NEWSLETTER_SUBSCRIBER` and anything else — they are intentionally standalone; a callback lead and a job applicant are different people with no guaranteed relationship, and forcing one would add complexity with no query benefit.

---

7. Entities

7.1 `Application`

The single table for **both** ASE Program and Job applications. See Section 8 for why one table is correct here.

Field	Type	Nullable	Notes
`id`	`VARCHAR(20)` PK	No	Human-readable ID, e.g. `APP202600001`. Generated server-side, never client-supplied.
`applicationType`	`ENUM`	No	`PROGRAM` | `JOB`
`job`	FK → `Job.id`	Yes	Set for `JOB` type. `NULL` for `PROGRAM`.
`jobTitleSnapshot`	`VARCHAR(150)`	Yes	Denormalized copy of the job title at time of application (`Program` sets this to `"Associate Software Engineer Program"`), so historical applications still display a title even if the `Job` row is later edited or deleted.
`fullName`	`VARCHAR(150)`	No	

email	VARCHAR(150)	No	Indexed.
phone	VARCHAR(20)	No	
college	VARCHAR(200)	Yes	PROGRAM only. NULL for JOB.
graduationYear	SMALLINT	Yes	PROGRAM only.
resumeUrl	VARCHAR(500)	No	Public/pre-signed URL, see Section 29.
resumeOriginalName	VARCHAR(255)	No	Original filename as uploaded.
resumeMimeType	VARCHAR(100)	No	
resumeSizeBytes	BIGINT	No	
resumeStorageKey	VARCHAR(500)	No	Internal object-store key, used to regenerate signed URLs; never sent to frontend.
note	TEXT	Yes	JOB only, optional field on ApplicationForm.jsx.
status	ENUM	No	See Section 9. Default PENDING (PROGRAM) or SUBMITTED (JOB).
isConsent	BOOLEAN	No	Default false. Set true only by the submit step.
submittedAt	DATETIME	Yes	Set when status transitions to SUBMITTED.
createdAt	DATETIME	No	Set on insert.
updatedAt	DATETIME	No	Set on every update.

7.2 Job

Field	Type	Nullable	Notes
id	BIGINT PK	No	Auto-increment.
slug	VARCHAR(100) UK	No	Matches frontend's job.id string, e.g. backend-developer. This is what the React router param actually is (/careers/jobs/:jobId), so the backend's public identifier must be the slug, not the numeric PK.
title	VARCHAR(150)	No	
department	VARCHAR(100)	No	
experience	VARCHAR(50)	No	Free text, e.g. "1 - 3 Years" (matches jobsData.js).
location	VARCHAR(150)	No	
employmentType	ENUM	No	FULL_TIME | INTERN_ASSOCIATE | CONTRACT — maps to the type badge shown on job cards.
description	JSON	No	Stores summary, aboutTeam, responsibilities[], requirements[], niceToHave[], perks[], tags[] as one JSON column — this shape is exactly the object literal already in jobsData.js, so moving it server-side is a lift-and-shift, not a redesign.
status	ENUM	No	OPEN | CLOSED | DRAFT
createdAt / updatedAt	DATETIME	No	

7.3 Lead

Field	Type	Nullable	Notes
id	BIGINT PK	No	
leadType	ENUM	No	CALLBACK | CONSULTATION
fullName	VARCHAR(150)	No	
company	VARCHAR(150)	Yes	CONSULTATION only.
email	VARCHAR(150)	No	Indexed.
phone	VARCHAR(20)	No	
projectType	VARCHAR(100)	Yes	CONSULTATION only.
budget	VARCHAR(50)	Yes	CONSULTATION only, stored as the display string (e.g. "$10k - $50k"), see note in Section 12.6.
message	TEXT	Yes	CONSULTATION only.
status	ENUM	No	Default NEW.

`createdAt` / `updatedAt`	`DATETIME`	No	

7.4 `NewsletterSubscriber`

Field	Type	Nullable	Notes
`id`	`BIGINT` PK	No	
`email`	`VARCHAR(150)` UK	No	Unique — re-subscribing an existing email reactivates it rather than erroring.
`status`	`ENUM`	No	`ACTIVE` | `UNSUBSCRIBED`
`subscribedAt`	`DATETIME`	No	
`unsubscribedAt`	`DATETIME`	Yes	

---

8. Relationships

- `Job` **1 → N** `Application` — one job posting can receive many applications. The FK is **nullable** because `PROGRAM`-type applications have no `Job` row (the ASE Program isn't listed in `jobsData.js`/the `Job` table — it's a distinct product, not a job requisition).
- `Lead` **and** `NewsletterSubscriber` **have no relationships** to `Application` or `Job` or each other. They represent a different kind of person (prospective customer vs. prospective employee vs. subscriber) and forcing a shared "Person" table at this stage would add join complexity for zero present benefit. Revisit only if/when a unified CRM view across leads *and* candidates becomes an actual requirement.

On the single `Application` table (you asked us to confirm this): Yes — **one table with an `applicationType` discriminator is correct here**, for three reasons:
1. **Same downstream consumer.** Whether a candidate came from the ASE Program or a job posting, HR/Admin ultimately manages them through the same pipeline (`PENDING → SUBMITTED → UNDER_REVIEW → ... → HIRED`). Two tables would mean the future Admin "All Candidates" screen has to `UNION` two tables on every list/search/filter query.
2. **Field overlap is high, divergence is low.** Only 3 fields differ (`college`, `graduationYear`, `isConsent`/`submittedAt` timing) — that's a case for nullable columns, not a second table. If the divergence grows significantly (e.g., Program adds 10 program-specific fields), revisit with a `JSON` "extra attributes" column rather than splitting the table.
3. **Marketing needs one funnel.** The PENDING-lead-recovery use case (Section 19) queries "everyone who started but didn't finish" — trivial as one `WHERE status = 'PENDING'` on one table, awkward across two.

The trade-off you're accepting: a few always-`NULL` columns for `JOB` rows. That's a fair price for a simpler read model.

---

9. Enums

```
public enum ApplicationType { PROGRAM, JOB }

public enum ApplicationStatus {
    PENDING,               // PROGRAM only — Step 1 done, Step 3 not yet
    SUBMITTED,              // consent given (PROGRAM) or single-step submit (JOB)
    UNDER_REVIEW,
    SHORTLISTED,
    INTERVIEW_SCHEDULED,
    SELECTED,
    REJECTED,
    HIRED
}

public enum JobStatus { OPEN, CLOSED, DRAFT }

public enum EmploymentType { FULL_TIME, INTERN_ASSOCIATE, CONTRACT }

public enum LeadType { CALLBACK, CONSULTATION }

public enum LeadStatus { NEW, CONTACTED, QUALIFIED, LOST, CONVERTED }

public enum NewsletterStatus { ACTIVE, UNSUBSCRIBED }
```

Why not `SUCCESS`, `FAILED`, etc. — those describe an HTTP outcome, not a recruitment state. The statuses above are the actual states HR reasons about, which is what a status column should encode.

Indexes:

Table	Index	Reason
`application`	`(application_type, status)` composite	Admin dashboard filters ("all pending program applications") and marketing recovery queries always filter on both.
`application`	`email`	Lookup / duplicate-check / support queries.
`application`	`created_at`	Pagination and "applications in last N days" reporting.
`application`	`job_id` FK index	Auto-created by FK, needed for "applications for this job" screen.
`lead`	`(lead_type, status)` composite	Same reasoning as applications — Admin/CRM filtering.
`lead`	`created_at`	Pagination/reporting.
`newsletter_subscriber`	`email` UNIQUE	Enforces one row per email; also the lookup path for subscribe/unsubscribe.
`job`	`slug` UNIQUE	Public lookup path is the slug, not the PK.
`job`	`status`	Public listing filters `status = OPEN` on every page load.

---

10. API List

Public (called by this React frontend)

Method	Path	Purpose
`GET`	`/api/v1/careers/jobs`	List open jobs
`GET`	`/api/v1/careers/jobs/{slug}`	Job detail
`POST`	`/api/v1/careers/program/applications`	ASE Program Step 1 — create `PENDING` application
`PATCH`	`/api/v1/careers/program/applications/{applicationId}/submit`	ASE Program Step 3 — consent + submit
`POST`	`/api/v1/careers/jobs/{slug}/applications`	Normal job application — single-step submit
`POST`	`/api/v1/contact/callback`	Request Callback lead
`POST`	`/api/v1/contact/consultation`	Schedule Consultation lead
`POST`	`/api/v1/newsletter/subscribe`	Newsletter subscribe

Future — Admin (Section 27 has full detail)

Method	Path	Purpose
`POST`	`/api/v1/admin/auth/login`	Admin login
`GET`	`/api/v1/admin/applications`	List/search/filter all applications
`GET`	`/api/v1/admin/applications/{id}`	Application detail
`PATCH`	`/api/v1/admin/applications/{id}/status`	Move through recruitment pipeline
`GET`	`/api/v1/admin/leads`	List/search/filter leads
`PATCH`	`/api/v1/admin/leads/{id}/status`	Update lead CRM status
`GET`	`/api/v1/admin/newsletter/subscribers`	List subscribers
`POST`	`/api/v1/admin/jobs`	Create job
`PUT`	`/api/v1/admin/jobs/{id}`	Edit job
`PATCH`	`/api/v1/admin/jobs/{id}/status`	Open/close/draft a job
`DELETE`	`/api/v1/admin/jobs/{id}`	Remove job (soft-delete recommended)

---

11. REST Endpoint Naming Conventions

- Base path is versioned: `/api/v1/...` — allows a breaking `v2` later without touching live frontend traffic.
- Resource names are **plural nouns**: `/jobs`, `/applications`, `/leads`. No verbs in the path.
- Actions that aren't plain CRUD are modeled as a **sub-resource verb via PATCH**, not a query param: `/applications/{id}/submit`, `/applications/{id}/status`, `/jobs/{id}/status`. This keeps the verb discoverable in the URL and

keeps HTTP method semantics honest (`PATCH` = partial update).

- Path identifiers use the **externally meaningful ID**: `job/{slug}` not `job/{numericId}`, because that's what the router already puts in the URL (`/careers/jobs/:jobId` in `JobPage.jsx` uses the string id from `jobsData.js`). `Application` uses its generated human-readable ID (`APP202600001`) for the same reason — it's what the frontend stores and later echoes back on the submit call.
- Nesting reflects real ownership only one level deep: `/careers/jobs/{slug}/applications` (an application belongs to a job) — but the **Program** application is *not* nested under a fictional job, it lives at `/careers/program/applications`, because the ASE Program is not a `Job` row (see Section 8).
- Query params are for filtering/pagination on `GET` collections: `?status=OPEN&page=0&size=20`.
- All request/response bodies are `camelCase` JSON, matching JS conventions on the frontend (no case-translation layer needed).

---

12. API Contracts

All responses use one envelope:

```
// Success
{
  "success": true,
  "message": "Human readable message",
  "data": { }
}

// Error
{
  "success": false,
  "message": "Human readable summary",
  "errors": [
    { "field": "email", "message": "must be a valid email address" }
  ],
  "timestamp": "2026-07-26T10:15:30Z",
  "path": "/api/v1/careers/program/applications"
}
```

`errors[]` is populated only for `400 VALIDATION_ERROR`; omitted otherwise.

12.1 `GET /api/v1/careers/jobs`

List all publicly visible jobs (`status = OPEN`).

Query Params: none required. `?department=Engineering` optional filter (future-friendly, not required for v1).

Response `200 OK`

```
{
  "success": true,
  "message": "Jobs fetched successfully.",
  "data": [
    {
      "slug": "backend-developer",
      "title": "Backend Developer",
      "department": "Engineering",
      "type": "Intern / Associate",
      "experience": "6 - 12 Months",
      "location": "Mumbai / Hybrid",
      "tags": ["Java", "Spring Boot", "MySQL", "REST APIs"]
    }
  ]
}
```

Note: the list endpoint returns only the **card-level fields** actually rendered on `CareersPage.jsx` (`role.type`, `role.experience`, `role.location`, `role.tags.slice(0,3)`). It intentionally omits `responsibilities`, `perks`, etc. — those load on `GET /jobs/{slug}` when the user opens a specific role, keeping the list payload light.

Errors: none expected beyond `500 INTERNAL_SERVER_ERROR`.

12.2 `GET /api/v1/careers/jobs/{slug}`

Full job detail — the data `JobPage.jsx`'s "details" step renders.

Path Param: `slug` — e.g. `qa-engineer`.

Response `200 OK`

```
{
  "success": true,
  "message": "Job fetched successfully.",
  "data": {
    "slug": "qa-engineer",
    "title": "QA Engineer",
    "department": "Quality Assurance",
    "type": "Intern / Associate",
    "experience": "6 - 12 Months",
    "location": "Mumbai / Hybrid",
    "tags": ["Manual Testing", "Automation", "Test Planning", "QA Tooling"],
    "summary": "...",
    "aboutTeam": "...",
    "responsibilities": ["..."],
    "requirements": ["..."],
    "niceToHave": ["..."],
    "perks": ["..."]
  }
}
```

Errors

Status	Condition	Body `message`
`404 NOT_FOUND`	slug doesn't exist, or job `status != OPEN`	`"Job not found."`

12.3 `POST /api/v1/careers/program/applications`

ASE Program **Step 1 — Basic Details**. Called the moment the user clicks **"Continue →"** on `ProgramApplicationForm.jsx`. Creates the record immediately so the lead is captured even if the user never returns.

Content-Type: `multipart/form-data`

Request Parts

Field	Type	Required	Validation
`fullName`	text	Yes	2–150 chars
`email`	text	Yes	valid email format, max 150
`phone`	text	Yes	10-digit numeric (matches the `+91` prefix shown in the UI)
`college`	text	Yes	2–200 chars
`graduationYear`	text/number	Yes	one of `[2022 .. 2027]` per `GRAD_YEARS` in `ProgramApplicationForm.jsx`
`resume`	file	Yes	`.pdf`, `.doc`, `.docx` only; max 5 MB (matches the frontend's stated limit)

Response `201 CREATED`

```
{
  "success": true,
  "message": "Application draft created successfully.",
  "data": {
    "applicationId": "APP202600001",
    "status": "PENDING"
  }
}
```

Frontend stores `applicationId` in component state (`applicationRef.current`), never displays it to the user.

Errors

Status	Condition	`message`
`400 VALIDATION_ERROR`	any field fails validation above	`"Validation failed."` + field errors
`413 PAYLOAD_TOO_LARGE`	resume exceeds 5 MB	`"Resume file exceeds the 5MB limit."`
`415 UNSUPPORTED_MEDIA_TYPE`	resume extension/MIME not in allow-list	`"Only PDF, DOC or DOCX files are accepted."`
`500 INTERNAL_SERVER_ERROR`	resume upload to object storage fails	`"Something went wrong. Please try again."`

Backend behavior: upload resume to object storage → generate `applicationId` → insert row with `applicationType=PROGRAM`, `status=PENDING`, `isConsent=false` → **no email sent** (Section 18 clarifies why).

12.4 `PATCH /api/v1/careers/program/applications/{applicationId}/submit`

ASE Program **Step 3 — Consent**. Called when the user clicks **"Submit Application →"** on `ConsentStep.jsx`, only enabled once all 4 checkboxes are checked.

Path Param: `applicationId` — the ID returned by 12.3.

Request

```
{
  "acceptedTerms": true
}
```

Validation
- `acceptedTerms` must be `true` (a `false`/missing value is a validation error — the frontend already disables the button until all 4 boxes are checked, but the backend must not trust that).
- The application referenced by `applicationId` must exist and currently be `status = PENDING`.

Response `200 OK`

```
{
  "success": true,
  "message": "Application submitted successfully."
}
```

Errors
Status	Condition	`message`
`404 NOT_FOUND`	no application with that ID	`"Application not found."`
`409 CONFLICT`	application is already `SUBMITTED` (or beyond) — prevents double-submit / resubmission, e.g. from a double-click or a stale browser tab	`"This application has already been submitted."`
`400 VALIDATION_ERROR`	`acceptedTerms` is not `true`	`"Consent is required to submit your application."`

Backend behavior: find by ID → update `status=SUBMITTED`, `isConsent=true`, `submittedAt=now()` → **trigger "Application Received" email asynchronously** → return. The email send must not block the HTTP response (Section 18).

12.5 `POST /api/v1/careers/jobs/{slug}/applications`

Normal job application — single step, from `ApplicationForm.jsx` via `JobPage.jsx`. `continueLabel="Submit Application"` on this form — this **is** the final step, unlike the generic `onContinue` naming suggests.

Content-Type: `multipart/form-data`

Path Param: `slug` — e.g. `frontend-developer`.

Request Parts
Field	Type	Required	Validation
`fullName`	text	Yes	2–150 chars (form field is `name` in the component state; backend/DTO field name is `fullName` for consistency with `Application.fullName` — see mapper note in Section 17)
`email`	text	Yes	valid email
`phone`	text	Yes	non-empty, matches `tel` input
`resume`	file	Yes	`.pdf` / `.doc` / `.docx`, max 5 MB
`note`	text	No	optional, matches the optional textarea

Response `201 CREATED`

```
{
  "success": true,
  "message": "Application submitted successfully."
}
```

Errors
Status	Condition	`message`
`404 NOT_FOUND`	`slug` doesn't match an `OPEN` job	`"This role is no longer accepting applications."`
`400 VALIDATION_ERROR`	missing/invalid required field	field-level errors
`413` / `415`	resume size/type	same as 12.3

Backend behavior: upload resume → insert row `applicationType=JOB`, `job=<resolved job>`, `jobTitleSnapshot=<job.title>`, `status=SUBMITTED` directly, `isConsent=false` (not applicable to this flow — left `false`, not `NULL`, since a Java `boolean` can't be `NULL`; see Section 7 field note), `submittedAt=now()` → trigger email async → return.

12.6 `POST /api/v1/contact/callback`

Request

```
{ "fullName": "Rahul Sharma", "email": "rahul@gmail.com", "phone": "9876543210" }
```

Validation: `fullName` 2–150 chars, `email` valid format, `phone` non-empty.

Response `201 CREATED`

```
{ "success": true, "message": "We've received your request. Our team will call you shortly." }
```

Errors: `400 VALIDATION_ERROR` on any field failure.

Backend behavior: insert `Lead` with `leadType=CALLBACK`, `status=NEW`. No email to the *customer* by default (a callback request implies a phone follow-up, not an email reply) — internal notification email to the sales inbox is a valid future addition, not required for v1.

12.7 `POST /api/v1/contact/consultation`

Request

```
{
  "fullName": "Rahul Sharma",
  "company": "ABC Pvt Ltd",
  "email": "rahul@gmail.com",
  "phone": "9876543210",
  "projectType": "Mobile App",
  "budget": "$5,000-$10,000",
  "message": "Need an Android and iOS application."
}
```

Validation: `fullName`, `email`, `phone` required as above; `company`, `projectType`, `budget`, `message` optional (the landing page form doesn't enforce them as required, so the backend shouldn't either — only guard against wildly oversized input, e.g. `message` max 2000 chars).

Note on `budget`: stored as the **display string** the `<select>` already produces (`"Under $10k"`, `"$10k - $50k"`, etc.) rather than parsed into a numeric range — the dropdown options are marketing-authored labels, not a structured range picker, and forcing a numeric range now is speculative. If Admin later needs numeric bucketing for reporting, that's a mapping table (`label → min/max`), not a schema change here.

Response `201 CREATED`

```
{ "success": true, "message": "Thanks! We'll be in touch to schedule your consultation." }
```

Backend behavior: insert `Lead` with `leadType=CONSULTATION`, `status=NEW`.

12.8 `POST /api/v1/newsletter/subscribe`

Request

```
{ "email": "rohan@gmail.com" }
```

Validation: valid email format.

Response `200 OK`

```
{ "success": true, "message": "Subscribed successfully." }
```

Backend behavior:
- Email not in table → insert, `status=ACTIVE`.
- Email exists, `status=UNSUBSCRIBED` → flip to `ACTIVE`, clear `unsubscribedAt`. **Do not error** — from the user's point of view they just subscribed.
- Email exists, `status=ACTIVE` → return the same success response idempotently (no duplicate row, no error) — a second click on a slow connection shouldn't surface an error to the user.

Errors: `400 VALIDATION_ERROR` only for a malformed email.

---

13. Controller Layer

Responsibility: HTTP concerns only — request mapping, `@Valid` triggering, building the `ResponseEntity` with the right status code, delegating everything else to a service. A controller method should read like a table of contents, not contain business logic.

```
@RestController
@RequestMapping("/api/v1/careers/program/applications")
@RequiredArgsConstructor
public class ProgramApplicationController {

    private final ProgramApplicationService service;

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<ApiResponse<ApplicationCreatedResponse>> create(
            @Valid @ModelAttribute ProgramApplicationCreateRequest request) {
        var result = service.createDraft(request);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(ApiResponse.success("Application draft created successfully.", result));
    }

    @PatchMapping("/{applicationId}/submit")
    public ResponseEntity<ApiResponse<Void>> submit(
            @PathVariable String applicationId,
            @Valid @RequestBody ProgramApplicationSubmitRequest request) {
        service.submit(applicationId, request);
        return ResponseEntity.ok(ApiResponse.success("Application submitted successfully.", null));
    }
}
```

No repository injections, no `try/catch` — exceptions bubble to `GlobalExceptionHandler` (Section 12's error table is implemented there).

---

14. Service Layer

Responsibility: the actual business rules — status transitions, validation that spans multiple fields, orchestration (upload resume, then insert, then queue email), and transaction boundaries (`@Transactional`).

```
@Service
@RequiredArgsConstructor
public class ProgramApplicationService {

    private final ApplicationRepository applicationRepository;
    private final ResumeStorageService resumeStorageService;
    private final ApplicationMapper mapper;
    private final ApplicationIdGenerator idGenerator;

    @Transactional
    public ApplicationCreatedResponse createDraft(ProgramApplicationCreateRequest request) {
        var resumeMeta = resumeStorageService.upload(request.getResume());
        var application = mapper.toEntity(request, resumeMeta);
        application.setId(idGenerator.next());
        application.setApplicationType(ApplicationType.PROGRAM);
        application.setStatus(ApplicationStatus.PENDING);
        application.setConsent(false);
        applicationRepository.save(application);
        return mapper.toCreatedResponse(application);
    }

    @Transactional
    public void submit(String applicationId, ProgramApplicationSubmitRequest request) {
        var application = applicationRepository.findById(applicationId)
                .orElseThrow(() -> new ResourceNotFoundException("Application not found."));

        if (application.getStatus() != ApplicationStatus.PENDING) {
            throw new InvalidStateException("This application has already been submitted.");
        }

        application.setStatus(ApplicationStatus.SUBMITTED);
        application.setConsent(true);
        application.setSubmittedAt(Instant.now());
        applicationRepository.save(application);

        emailService.sendApplicationReceivedEmailAsync(application); // fire-and-forget, see Section 18
    }
}
```

One service per module-owned use case cluster; `ProgramApplicationService` and `JobApplicationService` are separate classes even though they touch the same `Application` entity, because their business rules (two-phase vs. single-phase) genuinely differ — merging them into one `ApplicationService` with `if (type == PROGRAM)` branches would just move the module boundary problem inside a single class.

---

15. Repository Layer

Thin Spring Data JPA interfaces — no custom SQL beyond what derived queries or `@Query` can express cleanly.

```
public interface ApplicationRepository extends JpaRepository<Application, String> {

    Page<Application> findByApplicationTypeAndStatus(
            ApplicationType type, ApplicationStatus status, Pageable pageable);

    // Marketing recovery query — Section 19
    List<Application> findByApplicationTypeAndStatusAndCreatedAtBefore(
            ApplicationType type, ApplicationStatus status, Instant cutoff);

    boolean existsByEmailAndApplicationTypeAndStatus(
            String email, ApplicationType type, ApplicationStatus status);
}

public interface LeadRepository extends JpaRepository<Lead, Long> {
    Page<Lead> findByLeadTypeAndStatus(LeadType type, LeadStatus status, Pageable pageable);
}

public interface NewsletterSubscriberRepository extends JpaRepository<NewsletterSubscriber, Long> {
    Optional<NewsletterSubscriber> findByEmail(String email);
}

public interface JobRepository extends JpaRepository<Job, Long> {
    Optional<Job> findBySlugAndStatus(String slug, JobStatus status);
    List<Job> findByStatus(JobStatus status);
}
```

No repository method returns an entity directly to a controller — services always map to a DTO first (Section 17).

---

16. DTOs

Strict separation: **Entities never leave the service layer.** Every controller boundary crosses a DTO.

DTO	Direction	Used By
`ProgramApplicationCreateRequest`	in	12.3
`ProgramApplicationSubmitRequest`	in	12.4
`JobApplicationCreateRequest`	in	12.5
`CallbackRequest`	in	12.6
`ConsultationRequest`	in	12.7
`NewsletterSubscribeRequest`	in	12.8
`ApplicationCreatedResponse`	out	12.3
`JobResponse` / `JobSummaryResponse`	out	12.1 / 12.2
`ApiResponse<T>`	out (wrapper)	all

```
public record ProgramApplicationCreateRequest(
    @NotBlank @Size(min = 2, max = 150) String fullName,
    @NotBlank @Email @Size(max = 150) String email,
    @NotBlank @Pattern(regexp = "\\d{10}") String phone,
    @NotBlank @Size(min = 2, max = 200) String college,
    @NotNull @Min(2000) @Max(2100) Integer graduationYear,
    @NotNull MultipartFile resume
) {}
```

Bean Validation annotations live on the DTO, not the entity — the entity's constraints (`@Column(nullable = false)`) protect the database; the DTO's constraints protect the API contract. They can and should overlap, but they answer different questions.

---

17. Mapper Layer

MapStruct (or hand-written, equivalent effect) mappers convert DTO ⇄ Entity ⇄ Response DTO. Kept as plain interfaces/classes so the mapping logic is unit-testable in isolation from Spring context.

```
@Mapper(componentModel = "spring")
public interface ApplicationMapper {

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "status", ignore = true)
    @Mapping(target = "resumeUrl", source = "resumeMeta.url")
    @Mapping(target = "resumeStorageKey", source = "resumeMeta.storageKey")
    Application toEntity(ProgramApplicationCreateRequest request, ResumeMeta resumeMeta);

    ApplicationCreatedResponse toCreatedResponse(Application application);
}
```

One naming note carried through deliberately: the frontend's `ApplicationForm.jsx` local state field is `name`; every DTO and entity field in this document is `fullName`. The mapper (or the controller's `@ModelAttribute` binding) is the single place that reconciles this — see Section 31.2 for the recommended frontend-side fix instead, which is cleaner than permanently aliasing in the backend.

---

18. Email Flow

Trigger rule, stated once so it's unambiguous: an email is sent **only** when `Application.status` transitions to `SUBMITTED` — i.e., from `PATCH .../submit` (Program) or from `POST .../applications` (Job, which goes straight to `SUBMITTED`). **`POST program/applications` (Step 1 / `PENDING`) never sends an email.** This matches the reference doc's flow exactly and is the reason the marketing recovery use case (Section 19) is possible: a `PENDING` record with no email sent is exactly the audience for a "finish your application" nudge — sending a premature email would confuse a candidate who hasn't decided to apply yet.

Mechanism: `EmailService` methods are annotated `@Async`, backed by a dedicated thread pool (`AsyncConfig`), so the HTTP

response for `submit` / `applications` returns as soon as the DB write commits — the caller never waits on SMTP latency.

```
@Service
@RequiredArgsConstructor
public class EmailService {

    @Async("emailExecutor")
    public void sendApplicationReceivedEmailAsync(Application application) {
        var context = buildContext(application);
        var html = emailTemplateService.render(
            application.getApplicationType() == ApplicationType.PROGRAM
                ? "program-application-received"
                : "job-application-received",
            context
        );
        mailSender.send(application.getEmail(), "Application Received", html);
    }
}
```

Email content (Program): company logo, candidate name, program name, confirmation copy, "what's next" — mirrors exactly what `ProgramSuccessScreen.jsx` already shows on-screen, so the email and the success screen tell a consistent story.

Failure handling: an email send failure must **never** roll back the application submission — the DB transaction commits independently of the async email call. Failures are logged and (future) written to an `email_log` table with a `status` (`SENT` / `FAILED`) so a retry job or Admin screen can catch and resend failures, rather than silently losing them.

---

19. Marketing Flow

This is the direct payoff of the `PENDING` status (Section 8's reasoning #3):

```
-- "Started but never finished" — nightly job candidate query
SELECT * FROM application
WHERE application_type = 'PROGRAM'
  AND status = 'PENDING'
  AND created_at < NOW() - INTERVAL 24 HOUR
  AND created_at > NOW() - INTERVAL 30 DAY   -- don't re-nag ancient drafts forever
```

A scheduled job (`@Scheduled`, or an external cron hitting a protected internal endpoint) reads this set and sends a "Continue your application" email — distinct from and independent of the transactional "Application Received" email in Section 18. This is the concrete mechanism behind the reference doc's "100 clicked Continue, 30 clicked Submit → recover the other 70" scenario.

Newsletter subscribers (`status = ACTIVE`) are the second marketing audience — product updates, blog posts, hiring announcements, new service launches, per the original requirements. Both audiences are queried through their respective repositories; Section 23 covers where this logic eventually lives as the program matures beyond a single scheduled query.

---

20. Future Admin Flow

DIAGRAM (MERMAID SOURCE)

```
flowchart LR
    Admin[Admin Portal - separate app] -->|JWT| Gate[Auth Filter]
    Gate --> AC[AdminController]
    AC --> S1[ApplicationService]
    AC --> S2[LeadService]
    AC --> S3[JobService]
    AC --> S4[NewsletterService]
    S1 & S2 & S3 & S4 --> DB[(MySQL)]
```

Admin does **not** get its own copy of business logic — `AdminController` calls the *same* `JobService` / `ProgramApplicationService` /etc. used by the public controllers, just with admin-only DTOs (fuller field sets, pagination, filters) and behind an auth gate. This is why the module boundaries in Section 2 matter now, even though Admin doesn't exist yet: when it's built, it's new controllers + new DTOs over existing services, not new business logic.

Planned Admin capabilities (from the requirements): view/manage Jobs, all Applications (Pending + Submitted split), Program vs. Job candidates, Leads, Newsletter subscribers, update candidate status, open/close jobs.

---

21. Authentication Strategy (Future)

Not implemented in v1 — all current endpoints are public by design (a careers page and contact form can't require login). Documented now so Admin isn't bolted on awkwardly later.

- **Mechanism:** Spring Security + JWT (access token, short-lived, ~15 min; refresh token, longer-lived, rotated).
- **Scope:** only `/api/v1/admin/**` is secured. All current public endpoints stay open.
- **Roles:** `ROLE_ADMIN` (full access), `ROLE_HR` (applications/candidates only), `ROLE_MARKETING` (leads/newsletter only) — a simple `@PreAuthorize` split, not a full RBAC engine, unless the team grows enough to need one.
- **Password storage:** BCrypt, never reversible encryption.
- **Rate limiting:** login endpoint specifically, to blunt credential stuffing (e.g. bucket4j or a gateway-level limiter).
- **CORS:** the admin frontend origin is added to `CorsConfig` as its own allowed origin, separate from the public site's origin, so a misconfiguration on one can't silently loosen the other.

---

22. Future Scalability

Concern	Current Design	Scale Path
Resume storage	S3 (or equivalent), never in MySQL	Already scales independently of the DB; add a CDN in front of the bucket if resume *previews* become a frequent Admin action
Email sending	Async in-process thread pool	Swap to a message queue (SQS/RabbitMQ) + dedicated email worker once volume or reliability requirements outgrow an in-JVM thread pool — `EmailService`'s public method signature doesn't need to change, only its internals
Read-heavy endpoints (`GET /jobs`)	Simple JPA query	Add response caching (`@Cacheable`, short TTL) since job listings change rarely relative to how often they're read
Application volume	Single MySQL instance	Read replicas for Admin reporting queries once they're expensive enough to affect write-path latency; the write path (application submit) stays on the primary
Marketing recovery job	`@Scheduled` in the monolith	Extract to a separate scheduled worker/service once multiple scheduled jobs exist and you don't want them competing for the same JVM's resources
Module boundaries	Single Spring Boot app, package-per-module	Because modules don't cross-call each other's repositories (Section 2), any module can be extracted into its own service later without a rewrite — this is the actual reason the folder structure looks the way it does

---

23. Future Marketing Automation

Builds directly on Section 19:

- **Email Templates** as first-class entities (`EmailTemplate` table: `key`, `subject`, `htmlBody`, `updatedBy`) so Marketing can edit copy without a deploy.
- **Campaign** concept: a scheduled or triggered send to a *segment* (e.g., `NewsletterSubscriber WHERE status=ACTIVE`, or `Application WHERE type=PROGRAM AND status=PENDING`), with send/open/click tracking.
- **EmailEvent** — a lightweight internal publish point (`applicationSubmitted`, `leadCreated`, `subscriberJoined`) that both the transactional `EmailService` (Section 18) and future campaign triggers listen to, so adding a new automated email later is "subscribe a new listener," not "find every controller that needs to know about this."
- **Unsubscribe token:** every marketing (non-transactional) email includes a signed, single-use unsubscribe link hitting `POST /api/v1/newsletter/unsubscribe?token=...` — required for deliverability compliance (CAN-SPAM/GDPR-style expectations), not optional polish.

---

24. Sequence Diagrams

24.1 ASE Program

DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    actor U as Candidate
    participant FE as React (ProgramPage)
    participant BE as Spring Boot
    participant S3 as Object Storage
    participant DB as MySQL
    participant MAIL as Email Service

    U->>FE: Fills Basic Details, clicks Continue
    FE->>BE: POST /careers/program/applications (multipart)
    BE->>S3: upload resume
    S3-->>BE: resumeUrl
    BE->>DB: insert Application (PENDING)
    DB-->>BE: applicationId
    BE-->>FE: 201 { applicationId, status: PENDING }
    FE->>FE: store applicationId, go to Program Info step (no API call)
    U->>FE: Clicks "I Understand, Continue"
    FE->>FE: local step change only
    U->>FE: Checks all 4 boxes, clicks Submit Application
    FE->>BE: PATCH /careers/program/applications/{id}/submit
    BE->>DB: update status=SUBMITTED, isConsent=true, submittedAt
    BE-->>MAIL: sendApplicationReceivedEmailAsync() [fire-and-forget]
    BE-->>FE: 200 { success: true }
    FE->>U: ProgramSuccessScreen
    MAIL->>U: "Application Received" email
```

24.2 Normal Job Application

DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    actor U as Applicant
    participant FE as React (JobPage)
    participant BE as Spring Boot
    participant S3 as Object Storage
    participant DB as MySQL
    participant MAIL as Email Service

    U->>FE: Fills form, clicks "Submit Application"
    FE->>BE: POST /careers/jobs/{slug}/applications (multipart)
    BE->>S3: upload resume
    BE->>DB: insert Application (JOB, SUBMITTED)
    BE-->>MAIL: sendApplicationReceivedEmailAsync()
    BE-->>FE: 201 { success: true }
    FE->>U: SuccessScreen
    MAIL->>U: "Application Received" email
```

24.3 Schedule Consultation

DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    actor U as Visitor
    participant FE as React (LandingPage contact form)
    participant BE as Spring Boot
    participant DB as MySQL

    U->>FE: Fills consultation form, submits
    FE->>BE: POST /contact/consultation
    BE->>DB: insert Lead (CONSULTATION, NEW)
    BE-->>FE: 201 { success: true }
    FE->>U: inline confirmation message
```

24.4 Request Callback

DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    actor U as Visitor
    participant FE as React (LandingPage contact form)
    participant BE as Spring Boot
    participant DB as MySQL

    U->>FE: Fills callback form, submits
    FE->>BE: POST /contact/callback
    BE->>DB: insert Lead (CALLBACK, NEW)
    BE-->>FE: 201 { success: true }
    FE->>U: inline confirmation message
```

24.5 Newsletter Subscribe

DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    actor U as Visitor
    participant FE as React (Footer)
    participant BE as Spring Boot
    participant DB as MySQL

    U->>FE: Enters email, clicks subscribe
    FE->>BE: POST /newsletter/subscribe
    BE->>DB: find by email
    alt not found
        BE->>DB: insert (ACTIVE)
    else found, UNSUBSCRIBED
        BE->>DB: update status=ACTIVE
    else found, ACTIVE
        BE->>BE: no-op (idempotent)
    end
    BE-->>FE: 200 { success: true }
```

25. Why Every API Exists

API	Why it exists
`GET /careers/jobs`	Frontend currently hardcodes `JOBS` in `jobsData.js`; this endpoint is the seam that lets HR add/edit/close roles without a frontend deploy.
`GET /careers/jobs/{slug}`	Same reason, at detail-page granularity — also the endpoint that will one day return `status=CLOSED` and let the frontend show "this role is no longer open" instead of stale content.
`POST /careers/program/applications`	Captures the lead the instant Step 1 completes — the entire reason `PENDING` exists is so an abandoned application is still marketing-usable (Section 19). Without this endpoint firing on *Continue* rather than *Submit*, that data is lost forever the moment a candidate closes the tab.
`PATCH .../submit`	Separates "started" from "legally consented" — `isConsent=true` and the email must only fire once the candidate has actually agreed to terms, not before.
`POST /careers/jobs/{slug}/applications`	The job flow has no intermediate step to protect against — it's simpler by design (Section 8), so one endpoint does the whole job.
`POST /contact/callback` / `.../consultation`	Two separate endpoints (not one generic `/leads`) because their **required fields genuinely differ** (Section 12.6/12.7) — a single endpoint would need conditional validation logic that two clean endpoints avoid entirely.
`POST /newsletter/subscribe`	Smallest possible surface for a very common action; deliberately not folded into any other endpoint since it's reachable from many places (footer, future pop-ups) independent of any other flow.

26. Frontend Page → API Mapping

Frontend File	User Action	API Called
`ProgramApplicationForm.jsx` (via `ProgramPage.jsx`)	Click "Continue →"	`POST /careers/program/applications`
`ProgramInfoStep.jsx`	Click "I Understand, Continue →"	*none* — local state only
`ConsentStep.jsx` (via `ProgramPage.jsx`)	Click "Submit Application →"	`PATCH /careers/program/applications/{id}/submit`
`ApplicationForm.jsx` (via `JobPage.jsx`)	Click "Submit Application"	`POST /careers/jobs/{slug}/applications`
`CareersPage.jsx`	Page load	`GET /careers/jobs`
`JobPage.jsx` "details" step	Page load (route param `jobId`)	`GET /careers/jobs/{slug}`
`LandingPage.jsx` contact form, "Schedule Consultation" button	Click	`POST /contact/consultation`
`LandingPage.jsx` contact form, "Request Callback" button	Click	`POST /contact/callback`
`CareersLayout.jsx` / `LandingPage.jsx` footer form	Click subscribe (send icon)	`POST /newsletter/subscribe`

27. Future Admin APIs

Method	Path	Notes
`POST`	`/api/v1/admin/auth/login`	Returns access + refresh JWT
`POST`	`/api/v1/admin/auth/refresh`	Rotates access token
`GET`	`/api/v1/admin/applications?type=&status=&page=&size=`	Filterable, paginated list
`GET`	`/api/v1/admin/applications/{id}`	Full candidate detail, including resume signed URL
`PATCH`	`/api/v1/admin/applications/{id}/status`	`{ "status": "SHORTLISTED" }` — drives the recruitment pipeline
`GET`	`/api/v1/admin/leads?type=&status=&page=&size=`	Filterable, paginated list
`PATCH`	`/api/v1/admin/leads/{id}/status`	`{ "status": "CONTACTED" }`
`GET`	`/api/v1/admin/newsletter/subscribers?status=&page=&size=`	List/export
`POST`	`/api/v1/admin/jobs`	Create; body mirrors `Job` entity minus generated fields
`PUT`	`/api/v1/admin/jobs/{id}`	Full edit
`PATCH`	`/api/v1/admin/jobs/{id}/status`	`{ "status": "CLOSED" }` — the "Open/Close Jobs" requirement
`DELETE`	`/api/v1/admin/jobs/{id}`	Soft-delete recommended (`deletedAt` column) so historical applications keep a valid `job` reference

---

28. APIs the Frontend Will Never Call

These exist for internal orchestration, other services, or ops tooling — never invoked by this React app, and should be documented as such so nobody accidentally exposes them through the public CORS config.

- **Internal email webhook** (future) — bounce/complaint callbacks from the email provider (SES/SendGrid), used to mark subscribers/candidates as undeliverable.
- **Marketing recovery trigger** (Section 19) — either a `@Scheduled` method with no HTTP surface at all, or, if triggered externally by a cron service, an internal-only endpoint behind a shared secret / IP allowlist, never behind public CORS.
- **Resume signed-URL regeneration** — an internal service method (not necessarily even an endpoint) used by `AdminController` when it needs a fresh signed URL for `resumeStorageKey`; the public applicant flow never re-reads a resume it already uploaded.
- **Admin auth endpoints** — reachable from the future Admin Portal's origin only, not this public site's origin.
- **Health/metrics endpoints** (`/actuator/health`, `/actuator/prometheus`) — infrastructure/monitoring only.

---

29. Resume Storage Strategy

Recommendation: AWS S3 (or an S3-compatible provider — Cloudflare R2 works identically against the same SDK if cost is a concern later). Reasoning:

Option	Verdict
MySQL BLOB	Rejected — bloats backups, slows queries that touch the table, and every unrelated `Application` read pays for resume bytes it doesn't need.
Local disk	Rejected for anything beyond local dev — not durable across deploys/restarts on most hosting, doesn't survive horizontal scaling (two app instances, one disk each).
Cloudinary	Viable but built for images/video transformation; paying for that feature set to store PDFs is the wrong tool.
AWS S3	Correct fit — durable, cheap at this scale, decouples file storage from app deploys entirely, and every major hosting target (EC2, ECS, Fargate, even a plain VM) can reach it with just IAM credentials.

What the database stores (already reflected in Section 7.1): `resumeUrl` (what the frontend/admin can display — ideally a short-lived signed URL generated on read, not a permanently public one, since resumes are personal data), `resumeOriginalName`, `resumeMimeType`, `resumeSizeBytes`, and `resumeStorageKey` (the actual S3 object key, used internally to regenerate signed URLs — never returned to the frontend directly).

Upload validation, enforced server-side regardless of what the frontend already checks: extension allow-list (`.pdf`, `.doc`, `.docx`), MIME-type cross-check (don't trust the extension alone), 5 MB hard limit, and a virus/malware scan step is worth planning for once volume justifies it (e.g., ClamAV in the upload pipeline) — not required for v1 but noted so it isn't a surprise retrofit.

---

30. Best Practices & Naming Conventions

- **Java:** `PascalCase` classes, `camelCase` methods/fields, `UPPER_SNAKE_CASE` constants and enum values.
- **REST:** plural nouns, kebab-case only if a path segment is genuinely multi-word (none currently are), never camelCase or snake_case in the URL itself.
- **JSON:** `camelCase` keys everywhere, both request and response — no per-endpoint inconsistency.
- **DB columns:** `snake_case`, mapped via Hibernate's default naming strategy — Java stays `camelCase`, SQL stays idiomatic SQL.
- **Validation:** always at the DTO boundary via Bean Validation (`jakarta.validation.constraints`), never manually re-implemented `if (field == null)` chains in a service.
- **Exceptions:** one `GlobalExceptionHandler` (`@RestControllerAdvice`), each custom exception maps to exactly one HTTP status — no controller-level `try/catch` for expected error cases.
- **Transactions:** `@Transactional` at the service method level, never on a controller, never spanning an external I/O call (resume upload happens *before* opening the DB transaction, so a slow S3 call never holds a DB connection open).
- **Async:** never `@Async` a method that the caller needs the result of synchronously — email sending qualifies, resume upload does not (the frontend needs the `applicationId` back, which depends on the upload succeeding first).
- **Lombok:** `@Getter/@Setter` or `@Data` on entities/DTOs, `@RequiredArgsConstructor` for constructor injection — never field injection (`@Autowired` on a field) in new code.
- **Testing:** service-layer unit tests with mocked repositories; a smaller set of `@SpringBootTest` + Testcontainers (real MySQL) integration tests for the critical paths (program submit flow, job apply flow) rather than for every CRUD method.
- **API docs:** springdoc-openapi generates Swagger UI from the same annotated DTOs used for validation — this document is the authored source of truth, Swagger is the generated/interactive mirror of it, and the two should never be allowed to drift silently (review Swagger output whenever this document changes).

---

31. Frontend Code Observations

Three places where the current frontend code doesn't yet match the contract above. None of these block backend implementation — they're flagged so frontend work can be scheduled alongside it.

31.1 Contact form has two submit buttons with no distinguishing action

In `LandingPage.jsx`, the consultation `<form>` renders **two** `type="submit"` buttons ("Schedule Consultation" and "Request Callback") inside the *same* form, both currently only calling `e.preventDefault()`. As written, both buttons submit the same form data to whatever single handler exists — there's no way for the backend integration to know which of the two intents (and therefore which endpoint, 12.6 vs. 12.7) the user meant. **Recommended fix:** give each button its own `onClick` handler (`type="button"`, not `type="submit"`) that reads the current form state and calls the matching endpoint — `handleCallbackSubmit` posting only `{fullName, email, phone}` to `/contact/callback`, and `handleConsultationSubmit` posting the full payload to `/contact/consultation`.

31.2 `ApplicationForm.jsx` / `ProgramApplicationForm.jsx` use `name`, not `fullName`

Both forms keep local state as `form.name` (`ApplicationForm.jsx`) and similarly abbreviated keys (`form.mobile` vs. `phone`, `form.gradYear` vs. `graduationYear`). This document's DTOs use `fullName`, `phone`, `graduationYear` throughout for consistency with the `Application` entity. **Recommended fix:** rename the multipart field names sent on submit (not necessarily the internal React state) to match this contract exactly — e.g. `formData.append('fullName', form.name)` — so no translation layer is needed on the backend.

31.3 `JobPage.jsx` / `ProgramPage.jsx` TODOs assume two separate API calls

Both files' `TODO(backend)` comments describe submitting the application **and then** calling a separate `send-confirmation-email` endpoint. Per Section 18, the correct contract is **one call**: `submit` / `applications` triggers the email server-side, asynchronously, as part of the same request. **Recommended fix:** delete the second `fetch(...send-confirmation-email...)` call from both TODOs — the single `POST` / `PATCH` described in Sections 12.3–12.5 is sufficient and matches the "email happens once, only after SUBMITTED" rule you specified.

End of document.